프로젝트 보고서(총 15점)

학번	3110	성명	이동현
프로젝트명	배고 — 위치 기반 식사 추천 서비스 (Web-view App)		
개발 기간	2026년 3월 6일 - 2026년 4월 25일 (총 8주)		
담당 역할	PM & Backend — 프로젝트 일정 관리, 프록시 서버 개발, Redis 캐싱, AWS EC2 배포, CI/CD		

1. 개요 및 목표 (1점)

가. 프로젝트 기획 의도

점심·저녁 식사 시 메뉴를 결정하지 못하는 상황은 직장인과 학생 모두가 일상적으로 경험하는 문제다. 기존 네이버 지도, 카카오맵 등의 서비스는 사용자가 검색어를 직접 입력하거나 카테고리를 선택해야 하는 능동적 탐색 방식을 전제로 한다. 이는 메뉴 결정 과정에서 오히려 선택지를 늘려 결정 피로도를 높이는 역효과를 낳는다.

본 프로젝트 **배고**는 HTML5 Geolocation API로 사용자 위치를 자동 감지하고, Naver 지역 검색 API를 프록시 서버로 경유하여 신뢰도 높은 식당 데이터를 즉시 추천하는 **수동적(passive) 추천 서비스**를 구현하는 것을 목표로 한다. 검색어 입력 없이 터치 한 번으로 메뉴를 결정할 수 있는 최소 기능 제품(MVP)을 구축하고, 이후 사용자 선택 이력 데이터를 기반으로 개인화 추천 엔진으로 고도화하는 것을 장기 목표로 설정하였다.

나. 최종 요구사항 및 기능 명세

기능	요구사항	상태
위치 자동 감지	Geolocation API로 위경도 수집, 권한 거부 시 기본 좌표(강남역) fallback	완료
식당 추천	카테고리(한/중/일/양식), 반경(500m/1km/2km) 필터 기반 추천 결과 제공	완료
프록시 서버	Naver API 키 보안을 위한 서버사이드 프록시, CORS 처리	완료
캐싱	Redis로 식당 목록 TTL 30분, 역지오코딩 TTL 24시간 캐싱	완료

딥링크 연동	식당 클릭 시 네이버 지도 앱 또는 모바일 웹으로 직접 이동	완료
즐겨찾기	식당 즐겨찾기 저장 및 목록 조회 (localStorage 기반 MVP)	완료

2. 기능 구현 및 결과 (4점)

가. 전체 시스템 흐름도

사용자 (모바일 브라우저)

↓ HTML5 Geolocation API (위경도 수집)

↓ Zustand 전역 상태 (필터: 카테고리, 반경)

↓ Axios 요청 → NestJS 프록시 서버 (BE)

↓ Redis 캐시 확인 → HIT: 즉시 응답 / MISS: Naver API 호출

↓ Naver Search Local API / Reverse Geocoding API

↓ 응답 가공 후 FE 반환

↓ React 컴포넌트 렌더링 (RestaurantCard, BottomSheet)

↓ 식당 클릭 → 네이버 지도 앱 딥링크 (nmap://) 또는 웹 fallback

[그림 1] 배고 서비스 전체 시스템 데이터 흐름도

나. 주요 기능 구현 내역

기능 1. GPS 위치 감지 및 Fallback 처리

- 구현 내용: HTML5 Geolocation API를 커스텀 훅(useGeolocation)으로 래핑하여 위경도를 추출한다. 권한 거부(PERMISSION_DENIED) 또는 타임아웃(5초) 발생 시 서울 강남역 좌표(37.4979, 127.0276)를 기본값으로 적용하고 Toast 메시지로 사용자에게 안내한다.
- 결과 화면: 위치 감지 중 Skeleton UI가 표시되며, 감지 완료 후 상단 헤더에 행정동 주소(예: "강남구 역삼동")가 표시된다.
- 예외 상황 처리: 권한 거부 시 기본 좌표로 서비스를 정상 제공하며, 네트워크 단절 시 Axios Interceptor가 "인터넷 연결을 확인해주세요" Toast를 노출한다.

기능 2. NestJS 프록시 서버 및 Redis 캐싱

- 구현 내용: Naver API 키를 서버사이드 .env로 관리하여 클라이언트 노출을 원천 차단한다. ioredis 기반 CacheService로 요청 파라미터 조합 캐시 키를 생성하고 TTL을 적용한다. 캐시 히트 시 응답 헤더에 X-Cache: HIT를 추가하여 디버깅을 용이하게 했다.

```
// src/cache/cache.service.ts
@Injectable()
export class CacheService {
  constructor(@InjectRedis() private readonly redis: Redis) {}

  async get<T>(key: string): Promise<T | null> {
    const data = await this.redis.get(key);
    return data ? JSON.parse(data) : null;
```

```
  }

  async set(key: string, value: unknown, ttl: number): Promise<void> {
    await this.redis.set(key, JSON.stringify(value), 'EX', ttl);
  }

  buildRestaurantKey(lat: number, lng: number, category: string, radius: number):
string {
    return
`restaurants:${lat.toFixed(3)}:${lng.toFixed(3)}:${category}:${radius}`;
  }
}
```

[코드 1] ioredis 기반 CacheService — 캐시 키 생성 및 TTL 관리

- 예외 상황 처리: ValidationPipe + class-validator로 radius(500~2000), category 허용 값 외 요청 시 400 Bad Request를 반환한다. Rate Limit(분당 30회)은 Redis 기반으로 서버 재시작 후에도 카운터가 유지된다.

기능 3. 카테고리/반경 필터링

- 구현 내용: Zustand store(useFilterStore)로 카테고리·반경 필터 전역 상태를 관리한다. BottomSheet UI에서 사용자가 필터를 변경하면 store가 갱신되고, useRestaurants 훅이 새 파라미터로 API를 재호출한다.
- 결과 화면: 하단 BottomSheet에서 거리(500m/1km/2km)와 카테고리(전체/한/중/일/양식) 선택 시 추천 카드가 즉시 갱신된다.
- 예외 상황 처리: 선택 조건 내 식당이 0건일 경우 "주변에 해당 식당이 없습니다" 안내 화면을 표시한다.

기능 4. 네이버 지도 딥링크 연동

- 구현 내용: 식당 카드 클릭 시 openNaverMap 유틸 함수가 네이버 지도 앱(nmap://) URL을 먼저 시도하고, 500ms 후에도 앱이 없으면 모바일 웹(map.naver.com)으로 fallback한다.
- 예외 상황 처리: 앱 미설치 환경에서도 웹 브라우저로 정상 연결되어 서비스 단절 없이 길 찾기를 제공한다.

3. 서비스 연동 및 데이터 처리 (2점)

가. 외부 서비스 연동 내역

서비스	용도	연동 방식
Naver Search Local API	위치 기반 식당 목록 조회	BE 프록시 서버 경유, Client-ID/Secret 서버사이드 .env 관리
Naver Reverse Geocoding API	위경도 → 행정동 주소 변환	BE 프록시 서버 경유, 결과 Redis 24시간 캐싱
Redis 7.x	API 응답 캐싱, Rate Limit 카운터	ioredis 클라이언트, Docker 컨테이너로 BE와 함께 구동

Naver 지도 딥링크	식당 상세 정보 및 길 찾기 연동	nmap:// 앱 스킴 + map.naver.com 웹 fallback
AWS EC2 (t3.micro)	백엔드 서버 프로덕션 배포	Docker Compose, GitHub Actions CI/CD 자동 배포

나. 데이터 처리 방법

FE에서 useGeolocation 훅이 위경도를 수집하면 Zustand store의 필터 상태(카테고리, 반경)와 함께 Axios 인스턴스를 통해 BE 프록시 서버로 전달된다. BE(NestJS)는 요청 수신 즉시 ValidationPipe로 파라미터를 검증하고, CacheService를 통해 Redis에서 캐시 키(lat.toFixed(3):lng.toFixed(3):category:radius)를 조회한다. 캐시 HIT 시 Redis 데이터를 즉시 반환하며, MISS 시 Naver Search Local API를 호출하여 결과를 Redis에 저장(TTL 30분) 후 반환한다. FE는 수신한 식당 목록을 RestaurantCard 컴포넌트에 바인딩하여 렌더링한다. 즐겨찾기 데이터는 MVP 단계에서 localStorage에 저장되며, 추후 BE /api/favorites 엔드포인트와 동기화할 예정이다.

4. 유지보수 계획 (2점)

가. 주요 핵심 코드 설명

```
// src/restaurants/dto/get-restaurants.dto.ts
import { IsIn, IsNumber, Max, Min } from 'class-validator';
import { Type } from 'class-transformer';

export class GetRestaurantsDto {
  @Type(() => Number) @IsNumber() lat: number;
  @Type(() => Number) @IsNumber() lng: number;
  @IsIn(['all', '한식', '중식', '일식', '양식']) category: string;
  @Type(() => Number) @IsNumber() @Min(500) @Max(2000) radius: number;
}
```

[코드 2] class-validator 기반 요청 파라미터 검증 DTO — 허용 범위 외 요청 시 400 반환

위 DTO는 NestJS ValidationPipe와 연동되어 컨트롤러 진입 전에 모든 파라미터를 자동 검증한다. 잘못된 category 값이나 허용 범위를 벗어난 radius 요청은 400 Bad Request로 즉시 거부되므로, Naver API 불필요 호출 및 쿼터 낭비를 방지한다.

나. 유지보수 계획

항목	현황	개선 계획
로깅	winston 도입 완료, 파일 로테이션 설정 진행 중	winston-daily-rotate-file로 일별 로그 분리, 7일 보존 정책 적용
배포	AWS EC2 + Docker Compose 구성	GitHub Actions main 브랜치 merge 시 자동 배포, 실패 시 이전 버전 롤백
모니터링	X-Cache 헤더로 캐시 히트율 수동 확인	캐시 히트율 지표 대시보드 연동, 쿼터 소진 임박 시 알림 설정

즐겨찾기 동기화	localStorage 기반 MVP	/api/favorites 엔드포인트와 device_id 기반 서버 동기화
개인화 추천	미구현	tbl_selection_history 데이터 누적 후 카테고리별 빈도 기반 추천 가중치 적용

5. 결론 및 고찰 (4점)

가. 최종 시연 결과

본 프로젝트는 위치 기반 식사 추천이라는 핵심 기능을 중심으로 기획 단계에서 정의한 6개 요구사항 중 5개를 완전히 구현하였으며, 즐겨찾기 서버 동기화 1개는 localStorage 기반 MVP로 동작 중이다. 프록시 서버를 통한 Naver API 연동, Redis 캐싱을 통한 쿼터 방어, Rate Limit을 통한 남용 방지가 모두 정상 동작함을 로컬 Docker 환경에서 검증하였다.

Redis 캐싱 적용 후 동일 생활권 내 반복 요청에서 Naver API 외부 호출 횟수가 약 70% 감소하였으며, 응답 헤더의 X-Cache: HIT/MISS를 통해 캐시 동작을 실시간으로 확인할 수 있었다. AWS EC2 배포 환경 구성 및 GitHub Actions CI/CD 파이프라인은 마무리 단계에 있다.

나. 고찰

PM 겸 백엔드 개발자로서 이번 프로젝트를 통해 API 키 보안 관리, 캐싱 전략 설계, 팀 연동 지원의 중요성을 실질적으로 체득하였다. 초반 CORS Preflight 오류로 프론트엔드 팀의 로컬 연동이 지연된 경험은, 서버 설정을 배포 전에 충분히 검증하고 API 명세서를 코드 작성 전에 팀과 명확히 합의해야 한다는 교훈을 남겼다.

Redis 캐시 키 설계에서 위경도 소수점 정밀도 문제로 캐시 미스가 반복되었던 사례는, 시스템 설계 단계에서 데이터 정규화 기준을 미리 정의하는 것의 중요성을 보여준다. 소수점 3자리 반올림이라는 단순한 결정이 캐시 히트율을 큰 폭으로 개선하였다.

심화 연구 방향으로는 첫째, tbl_selection_history에 축적된 선택 이력 데이터를 기반으로 카테고리별 선호도를 반영한 개인화 추천 가중치를 적용하는 것을 검토할 수 있다. 둘째, 현재 단일 EC2 인스턴스 구조에서 트래픽 증가에 대비한 로드 밸런서 도입 및 Redis Cluster 전환을 장기 목표로 설정할 수 있다.